Diffusion Models and Confounder Free Neural Networks

George Hutchings
Supervisors: Prof Tom Nichols, Dr Habib Ganjgahi, Prof Chris Holmes
University of Oxford
Mini-project 2 Report

Tuesday 14th April, 2026

Table of Contents

1 Diffusion Models **2**
1.1 Introduction 2
1.2 Theory 2
1.2.1 The forward process 2
1.2.2 The reverse process 3
1.2.3 VAE interpretation 5
1.2.3.1 What is a VAE? 5
1.2.4 Score based generative modelling with differential equations (Song et al., 2021; Kreis et al., 2022) 6

2 Implementation of CF network **8**
2.1 Introduction 8
2.2 Implementation 8
2.2.1 The confounder prediction module 9
2.2.2 Training 9

3 Results **11**
3.1 On 20% of the data 11
3.2 On 100% of the data 11
3.3 With different confounders 12
3.4 Refining the model 13

1 | Diffusion Models

1.1 Introduction

Diffusion models have seen success in text to image generation (Nichol et al., 2021), image variations, image interpolation (Ramesh et al., 2022), super resolution (Saharia, Ho, et al., 2022), colourisation, inpainting, uncropping, jpeg restoration, (Saharia, Chan, et al., 2022), semantic segmentation (Baranchuk et al., 2021), video generation (Ho et al., 2022), solving the inverse problem in medical imaging (ie.undersampled MRI images) (Song et al., 2021), and many more.

Broadly, diffusion models attempt to generate data, usually images. To do this they are first trained on a dataset of images, and then given a random noise vector they generate an image. The training process is such that the model learns to generate images that are similar to the images in the training set. This brief chapter on diffusion models in this chapter is largely based on Weng (2021) and Kreis et al. (2022).

1.2 Theory

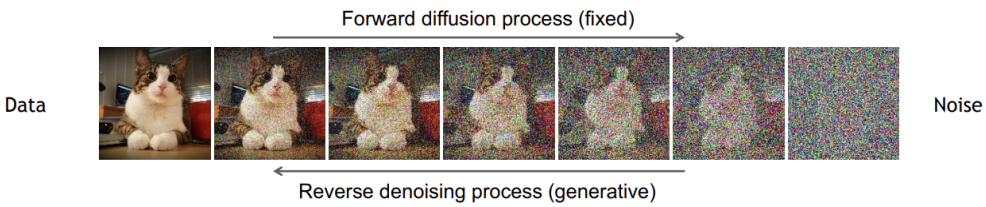

Figure 1.1: Simple figure showing the forward and reverse diffusion process, from Kreis et al. (2022).

1.2.1 The forward process

The forward process, as seen in figure 1.1 is the process of 'adding noise' to the data, this is done recursively, and in such a way that eventually the data becomes a sample from an isotropic Gaussian distribution. And so generation is then simply sampling from this isotropic Gaussian and then reversing these noise adding steps.

More formally, let $q(x)$ be the distribution of the data, and x_0 a data point sampled from such distribution. Let $x_1, x_2, \ldots, x_T$ correspond to the sequence of noisy samples (increasing in noise) obtained by adding noise to x_0, as seen in the slightly more annotated figure 1.2 And hence the conditional distribution of x_t given x_{t-1} is given by:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t|\sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

Where $\beta_t \in (0, 1)$ is the variance schedule, which controls the amount of noise added at each step. Therefore the joint distribution of $x_{1:T}|x_0$ is given by:

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$$

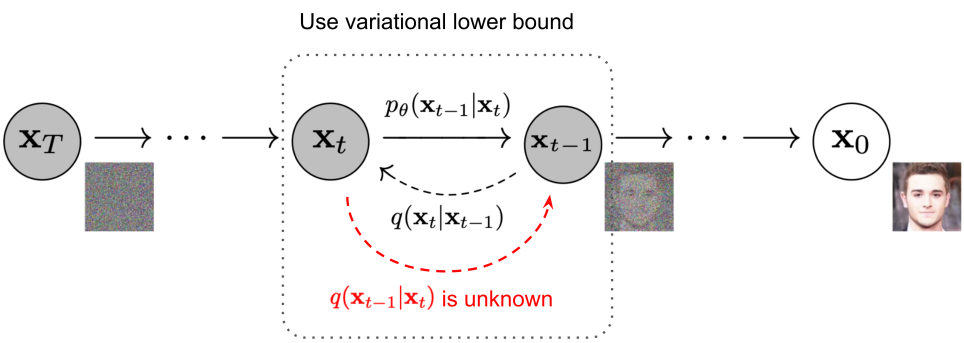

Figure 1.2: Annotated figure showing the forward and reverse diffusion process, from Weng (2021).

But does that mean to sample x_t given only x_0 one needs to sample $x_1, x_2, \ldots, x_{t-1}$? No, using the reparameterisation trick (the reparameterisation trick rewrites a random variable as a deterministic function of a noise variable (Luo, 2022)) the conditional distribution of x_t given x_0 can be calculated. The idea of the working is shown below, see Weng (2021) for full details.

$$
\begin{aligned}
x_t =& \sqrt{1-\beta_t} x_{t-1} + \sqrt{\beta_t}\epsilon_t \\
x_t =& \sqrt{1-\beta_t}\left(\sqrt{1-\beta_{t-1}} x_{t-2} + \sqrt{\beta_{t-1}}\epsilon_{t-1}\right) + \sqrt{\beta_t}\epsilon_t \\
=& \sqrt{1-\beta_t}\sqrt{1-\beta_{t-1}} x_{t-2} + \sqrt{1-\beta_t}\sqrt{\beta_{t-1}}\epsilon_{t-1} + \sqrt{\beta_t}\epsilon_t \\
& \ldots
\end{aligned}
$$

Which gives us the following distribution:

$$
q(x_t|x_0) = \mathcal{N}(x_t|\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I) \tag{1.1}
$$

Where $\bar{\alpha}_t = \prod_{i=1}^{t}(1-\beta_i)$.

1.2.2 The reverse process

The reverse process consists of recursively removing noise from x_T which is distributed approximately $\mathcal{N}(0, I)$, to eventually obtain the data, x_0. This is done by recursively removing the noise added in the forward process, ie. by sampling from the conditional distribution of x_{t-1} given x_t. Therefore we seek $q(x_{t-1}|x_t)$, which for suitable β_t is normal, however this is not tractable. Instead we use the following approximation $p_\theta(x_{t-1}|x_t)$, where θ are the parameters of the model.

$$
p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}|\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \tag{1.2}
$$

Where $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ are the mean and covariance of the distribution respectively. These are obtained by using a neural network to map x_t and t to $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$. We

can use the variational lower bound to maximise the negative log likelihood:

$$\begin{aligned}
-\log p_\theta(x_0) &\leq -\log p_\theta(x_0) + D_{KL}(q(x_{1:T}|x_0)||p_\theta(x_{1:T}|x_0)) \\
&= -\log p_\theta(x_0)\mathbb{E}_{q(x_{1:T}|x_0)}\left[\log \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0)}\right] \\
&= -\log p_\theta(x_0)\mathbb{E}_{q(x_{1:T}|x_0)}\left[\log \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T})/p_\theta(x_0)}\right] \\
&= \mathbb{E}_{q(x_{1:T}|x_0)}\left[\log \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T})}\right]
\end{aligned}$$

Where D_{KL} is the Kullback-Leibler divergence, hence:

$$-\mathbb{E}\left[\log p_\theta(x_0)\right] \leq \mathbb{E}_{q(x_{0:T})}\left[\log \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T})}\right] := \mathcal{L}$$

It can be shown (Ho et al., 2020) that:

$$\begin{aligned}
\mathcal{L} &= \mathbb{E}_q\left[\log \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T})}\right] \\
&\cdots \\
&= \sum_{t=0}^{T} L_t
\end{aligned}$$

where

$$\begin{aligned}
L_0 &= -\log(p_\theta(x_0|x_1)) \\
L_t &= D_{KL}(q(x_t|x_{t+1}, x_0)||p_\theta(x_t|x_{t+1})),\ t = 1, 2, \ldots, T-1 \\
L_T &= D_{KL}(q(x_T|x_0)||p_\theta(x_T))
\end{aligned}$$

Hence

$$\min_\theta \mathcal{L} = \min_\theta \sum_{t=0}^{T-1} L_t$$

As x_T is Gaussian noise, hence $p_\theta(x_T)$ is Gaussian noise, therefore L_T is constant and can be ignored.

$q(x_{t-1}|x_t, x_0)$ is tractable and can be shown to be:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}|\tilde{\mu}(x_t, t), \tilde{\Sigma}_t)$$

Where:

$$\begin{aligned}
\tilde{\mu}(x_t, x_0) &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_t\right) \\
\tilde{\Sigma}_t &= \tilde{\beta}_t I \\
\alpha_t &= 1 - \beta_t \\
\tilde{\beta}_t &= \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t
\end{aligned}$$

Hence we need to only take μ_θ of the form $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$, where $\epsilon_\theta(x_t, t)$ is a neural network, then:

$$L_t = \frac{1}{2}\mathbb{E}_{x_0}\left[\frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)\|\Sigma_\theta\|^2}\|\epsilon_t - \epsilon_\theta(x_t, t)\|^2\right] \tag{1.3}$$

Where Ho et al. (2020) suggest using a simplified version instead, and show that empirically it performs well.

$$L_t^{simple} = \mathbb{E}_{x_0}\left[\|\epsilon_t - \epsilon_\theta(x_t, t)\|^2\right] \tag{1.4}$$

This gives the training and sampling algorithm as seen in figure 1.3.

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$\nabla_\theta \left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)\right\|^2$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Figure 1.3: Algorithm for backward and forward diffusion process, from Weng (2021).

1.2.3 VAE interpretation

Diffusion models can be thought of as a VAE.

1.2.3.1 What is a VAE?

The VAE interpretation of the model is as follows. The encoder is the forward process, which maps the data to a latent space. The decoder is the reverse process, which maps the latent space to the data. This can be seen in the graphical model in figure 1.4. Figure 1.4, shows a VAE with only one latent component, consider for a moment a VAE with multiple latent components, in a hierarchical fashion, as shown in figure 1.5.

And a generative model of the form:

$$p_\theta(x, z_1, z_2) = p_\theta(x|z_1)p(z_1|z_2)p(z_2)$$

say we seek to maximise the log likelihood of the data $p_\theta(x)$, then we can use the variational lower bound:

$$\begin{aligned}
\log p_\theta(x) &\geq \mathbb{E}_{z_1, z_2 \sim q_\psi(z_1, z_2|x)}\left[\log \frac{p_\theta(x, z_1, z_2)}{q_\psi(z_1, z_2|x)}\right] \\
&= \mathbb{E}_{z_1, z_2 \sim q_\psi(z_1|x)q_\psi(z_2|z_1)}\left[\log \frac{p_\theta(x|z_1)p(z_1|z_2)p(z_2)}{q_\psi(z_1|x)q_\psi(z_2|z_1)}\right] \\
&= \mathbb{E}_{q(z_1|z_2)}\left[\log p_\theta(x|z_1)\right] - D_{KL}(q_\psi(z_1|x)||p_\theta(z_1|x)) - D_{KL}(q_\psi(z_2|z_1)||p_\theta(z_2))
\end{aligned}$$

This looks familiar to the loss function optimised in the diffusion models section, ie figure 1.6. The difference being in the inference model q there are no learnable parameters,

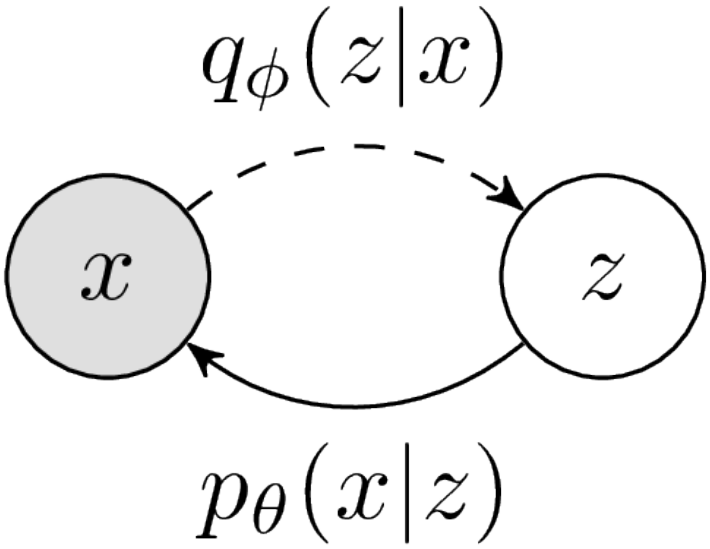

Figure 1.4: Graphical model of a VAE, from Turner (2021).

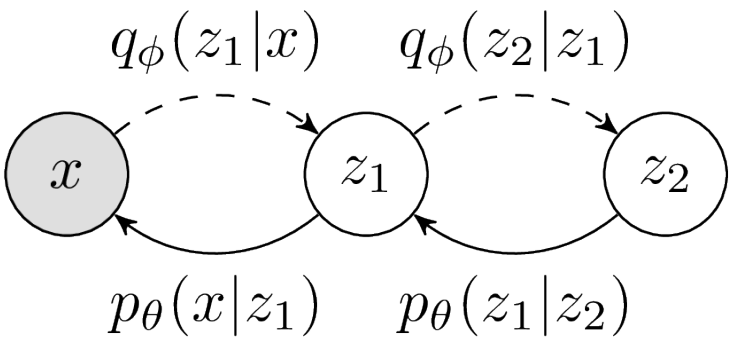

Figure 1.5: Graphical model of a hierarchical VAE, from Turner (2021).

and is constructed so that the final latent distribution $q(x_T)$ is approximately a standard Normal distribution. Additionally the latent space is of the same size as the input space.

1.2.4 Score based generative modelling with differential equations (Song et al., 2021; Kreis et al., 2022)

Diffusion models can be formulated as a stochastic differential equation featuring the score function.

- Forward Diffusion SDE:

$$dx_t = -\frac{1}{2}\frac{\beta_t}{\nabla t}x_t dt + \sqrt{\frac{\beta_t}{\nabla t}}d\omega_t$$

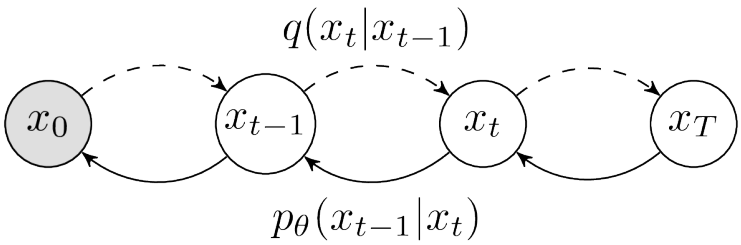

Figure 1.6: Diffusion Probabilistic model, from Turner (2021).

- Reverse Diffusion SDE:

$$dx_t = -\frac{1}{2}\frac{\beta_t}{\nabla t}x_t - \frac{\beta_t}{\nabla t}\nabla_{x_t}\log q_t(x_t)dt + \sqrt{\frac{\beta_t}{\nabla t}}d\omega_t$$

The score function is the gradient of the log probability of the data, and is approximated using score matching techniques.

In distribution these SDEs are equivalent to the probability flow ODE:

$$dx_t = -\frac{1}{2}\frac{\beta_t}{\nabla t}\left(x_t + \nabla_{x_t}\log q_t(x_t)\right)dt + \sqrt{\frac{\beta_t}{\nabla t}}dt$$

Hence to solving this ODE results in the same $q(x_t)$ as solving the SDE when initialising $q_T(x_T) \approx \mathcal{N}(0, I)$.

2 | Implementation of CF network

2.1 Introduction

The confounder free network attempts to remove the correlation between a given set of confounders and the feature space without a neural network. In principal it can be applied to any number of confounders and an arbitrary neural network. In this project we will focus on the case of three confounders, and a U-net with the structure seen in the appendix, this structure is identical to that used by Yang.

The inspiration for this type of model is drawn from work by Dinsdale et al. (2021); Zhao et al. (2020).

2.2 Implementation

Consider a vanilla U-net, which aims to segment lesions, it has a in our case 4 input channels and a single output channel. The input channels T1, T2, proton density and pseudo FLAIR, and the output channel is the prediction. Let us split this single U-net into two networks F and P where F consists of all layers except the final layer and P which consists of only the final layer, the original network can be recovered by the composition of F and P.

On the other hand the confounder free network is identical to the vanilla network except it has an additional module which attempts to predict the confounders. This module takes as input the feature space, which is the output of F and outputs the confounders[1]. The architecture of this is seen in figure 2.1. And the relationship between the confounders and the features is attempted to be removed.

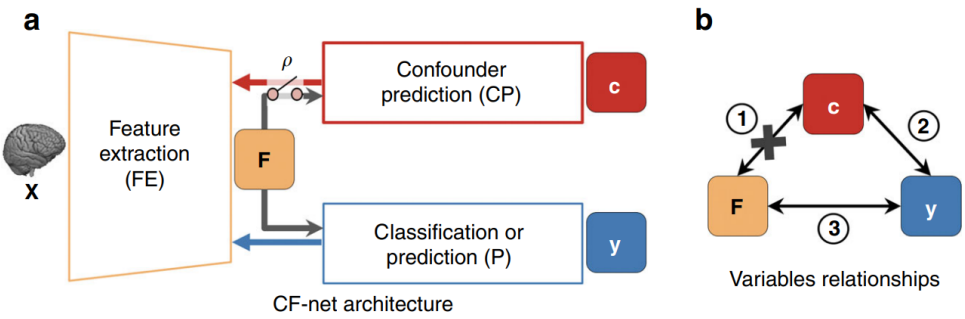

Figure 2.1: Confounder free network architecture, from Zhao et al. (2020).

This relationship is removed by using the following loss functions.

$$L_p = l(\hat{y}_i, y_i)$$

$$L_c = -\sum_{i=1}^{K} corr^2(c^i, \hat{c}^i)$$

[1]More rigorously, the confounder free network attempts to only predict items that are highly correlated with the confounder, but for simplicity this is ignored for now.

Where $\hat{y}_i$ is the prediction of the network, and y_i is the ground truth, c^i is the ith confounder and $\hat{c}^i$ is the prediction of that confounder. K is the number of confounders. l is the loss function used for the segmentation, in this case it is a combination of dice loss and binary cross entropy. The correlation is calculated using the Pearson correlation coefficient. Note, perhaps $L_c = -\frac{1}{K}\sum_{i=1}^{K} corr^2(c^i, \hat{c}^i)$ would be better as then the loss doesn't scale with the number of confounders.

Zhao et al. (2020) recommend training the network as follows:

$$\min_{\theta_F, \theta_P} \max_{\theta_C} L_p - \lambda L_c$$

Where θ_F, θ_P are the parameters of F and P respectively, and θ_C are the parameters of the confounder prediction module. λ is a hyperparameter which controls the tradeoff between the two losses. That is, model F minimises the segmentation loss and maximises the correlation loss (equivalent to minimising the squared correlation), model P minimises the segmentation loss, and the confounder prediction module minimises the correlation loss (attempts to predict the confounders as accurately as possible). The idea is to obtain a feature space that is uncorrelated with the confounders, while retaining important information necessary for the segmentation task.

2.2.1 The confounder prediction module

The confounder prediction module consists of the corresponding down convolution layers to bring the dimensionality back down to the size of the bottleneck, following this the data is flattened and there are two fully connected layers, with a RELU activation in between and after the final hidden layer, where the first hidden layer has 16 neurons (and the second K). A detailed image of the architecture cn be seen in figure 7 in the appendix. This structure follows that of Dinsdale et al. (2021). Including the location of where the confounder prediction module is placed in the network. The reason the module is not placed earlier in the network is because otherwise after the confounder prediction module has attempted to remove the correlation between the confounders and the feature space, the segmentation network could then attempt to re-introduce this correlation through the skip connections in the U-net (Dinsdale et al., 2021).

2.2.2 Training

For training the network the parameters of each model are updated separately in the following order:

1. θ_F is trained by minimising $L_p - \lambda L_c$.
2. θ_C is trained by minimising λL_c.
3. θ_P is trained by minimising L_p.

This is slightly different to Zhao et al. (2020) where the following update order is used:

1. θ_F, θ_P is trained by minimising L_p.
2. θ_C is trained by minimising λL_c.
3. θ_F is trained by minimising $-\lambda L_c$.

The reason why the former order is used, is because when implementing into pytorch it is far more more efficient. The architecture of the models can be seen in the appendix, in figures 5, 6, 7

3 | Results

The data is broadly split into three categories, this is done by Yang.

- Training data: used to train the model, consists of 5000 MRI scans
- Seen test data (held out) used to calculate metrics, consists of 5218 MRI scans. Contains data that differs from the training data by only one of scanner or scanner protocol.
- Unseen test data (held out) calculate metrics, consists of 5218 MRI scans. Contains data that differs from the training data by both of scanner or scanner protocol.

Additionally preprocessing of the slices follows the same procedure as Yang, with scans that do not correspond to any confounders omitted. Initially the confounders used are the UMAP components 1 to 3.

3.1 On 20% of the data

To begin the model was trained on only 20% of the data, using ADAM optimiser with initial learning rate of 0.0001, with cosine annealing learning rate scheduler. The model was trained for 150 epochs with a batch size of 128, and with $\lambda = 1$. Interestingly the vanilla model converged within approximately 30 epochs. While the confounder free model appeared to converge within only 30 epochs, with little change in L_c throughout, however with extended training, the model decreased its L_c at about 110 epochs, converging at 150 epochs. This is perhaps due to the difficulty training with the adversarial confounder module. When comparing the performance of the model in the vanilla and confounder free models perform similarly. As seen below in table 3.1.

prec mean	prec std	recall mean	recall std	dice mean	dice std	model	Severity
0.599	0.248	0.563	0.295	0.528	0.263	CF	Baseline
0.668	0.226	0.525	0.296	0.527	0.272	V	Baseline
0.597	0.248	0.561	0.296	0.526	0.263	CF	Moderate
0.664	0.228	0.52	0.296	0.522	0.271	V	Moderate
0.455	0.253	0.728	0.265	0.513	0.247	CF	Severe
0.527	0.252	0.681	0.286	0.545	0.25	V	Severe

Table 3.1: Table showing metrics for the model trained on 20% of data.

3.2 On 100% of the data

Following this, with an identical set up to training on 20% of the data the model was trained on 100% of the data. However the confounder loss remained at its minimum (predictions completely correlated with the confounders), even when extending training to 300 epochs. Yang suggests this may be due to incorrect learning rate, and further tuning should be attempted. The results can be seen in table 3.2. Here, again very similar performance is seen between the confounder free and vanilla models and, in the vanilla model the pattern seen is identical to that observed in the U-net by Yang.

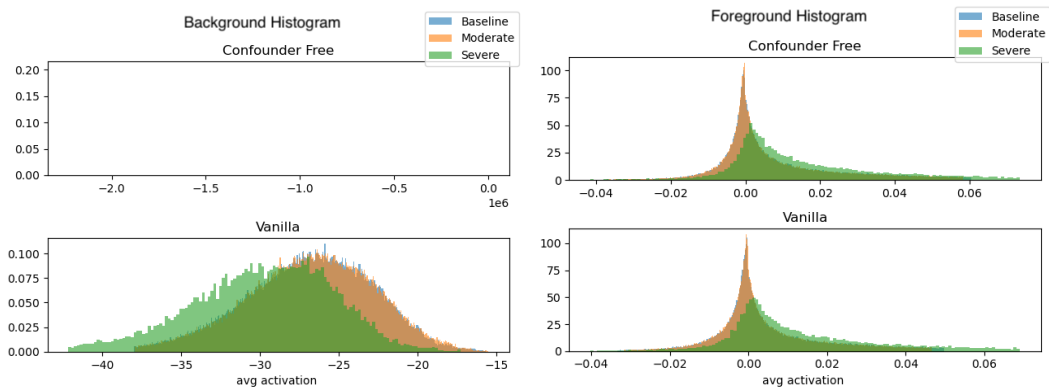

Figure 3.1: Histogram of the mean last layer of activations for the model trained on 20% of data, note the confounder free histogram was not able to plot due to the massively different scale between some severe observations and the others.

prec mean	prec std	recall mean	recall std	dice mean	dice std	model	Severity
0.712	0.184	0.63	0.266	0.628	0.244	CF	Baseline
0.7	0.186	0.634	0.27	0.624	0.244	V	Baseline
0.663	0.211	0.585	0.272	0.576	0.246	CF	Moderate
0.655	0.213	0.592	0.274	0.575	0.246	V	Moderate
0.474	0.232	0.755	0.248	0.545	0.227	CF	Severe
0.479	0.236	0.748	0.255	0.544	0.23	V	Severe

Table 3.2: Table showing metrics for the model trained on 100% of data.

3.3 With different confounders

Perhaps the UMAP components are not the most appropriate confounders to use. To evaluate this consider the histograms of the UMAP components, as well as several other confounders. As seen in figure 3.3 the UMAP components have several outliers that are likely affecting the model and so perhaps are not the most appropriate confounders to use.

Consider using CJV, CNR, total SNRd and total SNR as confounders. One can see histograms of these in figure 3.4. CJV, CNR, total SNRd provide nicer distributions than the UMAP components, and also show clear differences between severe vs baseline and moderate. (though this is not the case for total SNR) And hence perhaps these would be better confounders to use.

The model is trained with these confounders, though the precision, recall and dice scores have not yet been calculated due to time constraints. Just considering the validation loss (L_c, L_p), one can see that the model's confounder prediction loss L_c, like with the previous model that was trained on 100% of the data, does not increase (even when training for 300 epochs). The L_p loss does decrease and seems competitive with the previous model, however without considering metrics on held out data it is difficult to say whether this model is better or worse than the previous model.

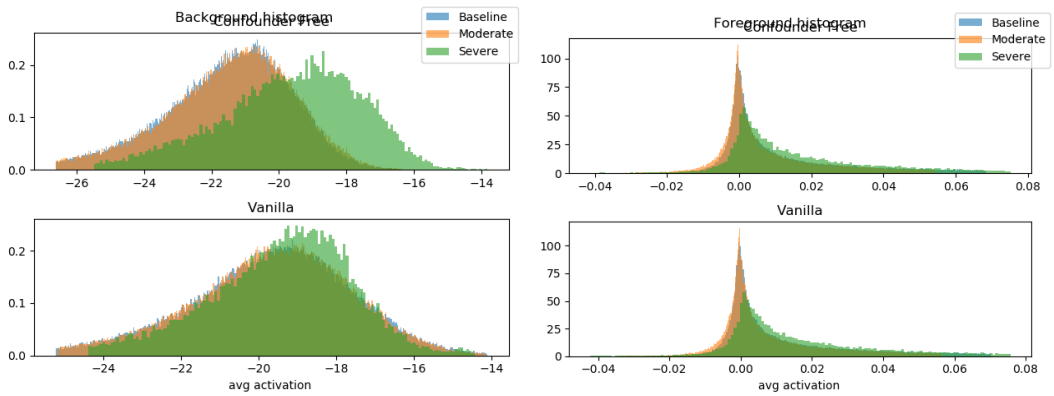

Figure 3.2: Histogram of the mean last layer of activations for the model trained on 100% of data, as seen the confounder free network does not bring the distributions closer.

3.4 Refining the model

To attempt to refine the model I decrease the learning rate to 0.000001 and remove the cosine scheduler, in addition to this I change $\lambda = 10$ in attempt to place more weight on the confounder predicting model, additionally I increased the batch size to 192 in an attempt to increase the speed. This model is still training, however the results look almost identical to the previous model, with the confounder loss not decreasing. And potentially the confounder predicting model is too complex and therefore is always able to predict the confounders well, though perhaps reducing the learning rate further will help.

All the code to train the model and produce the results can be found on the cluster at `/home/uu85g9/confounder_free_nn/my_unet`.

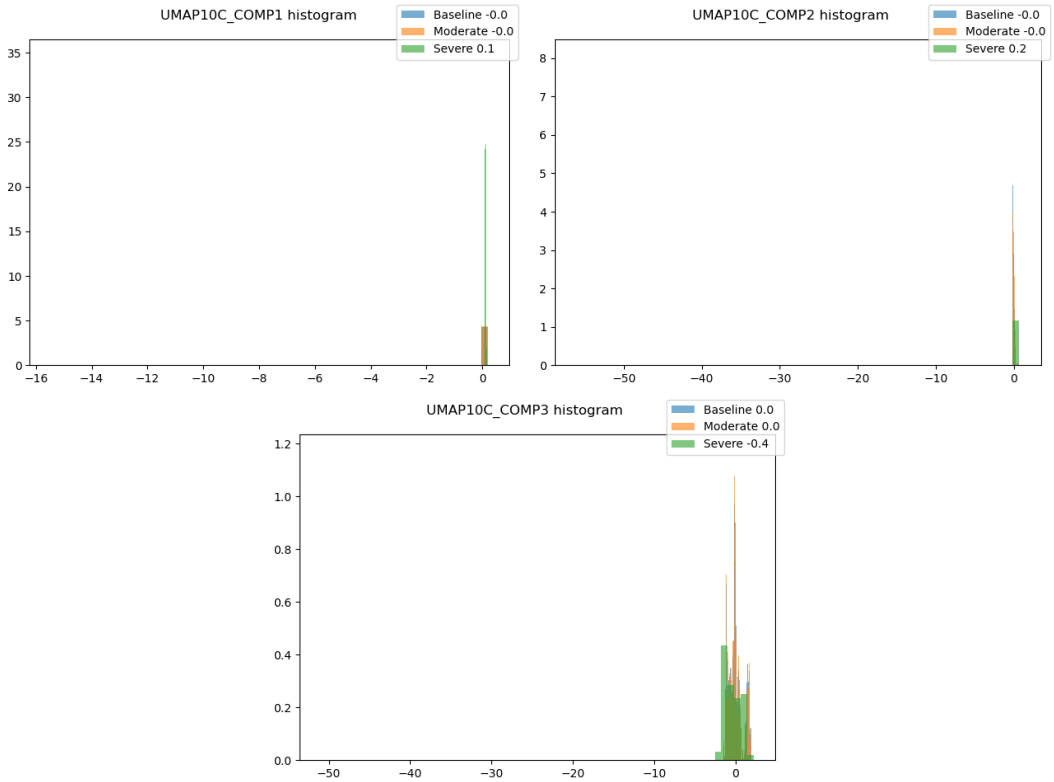

Figure 3.3: Histograms of UMAP components.

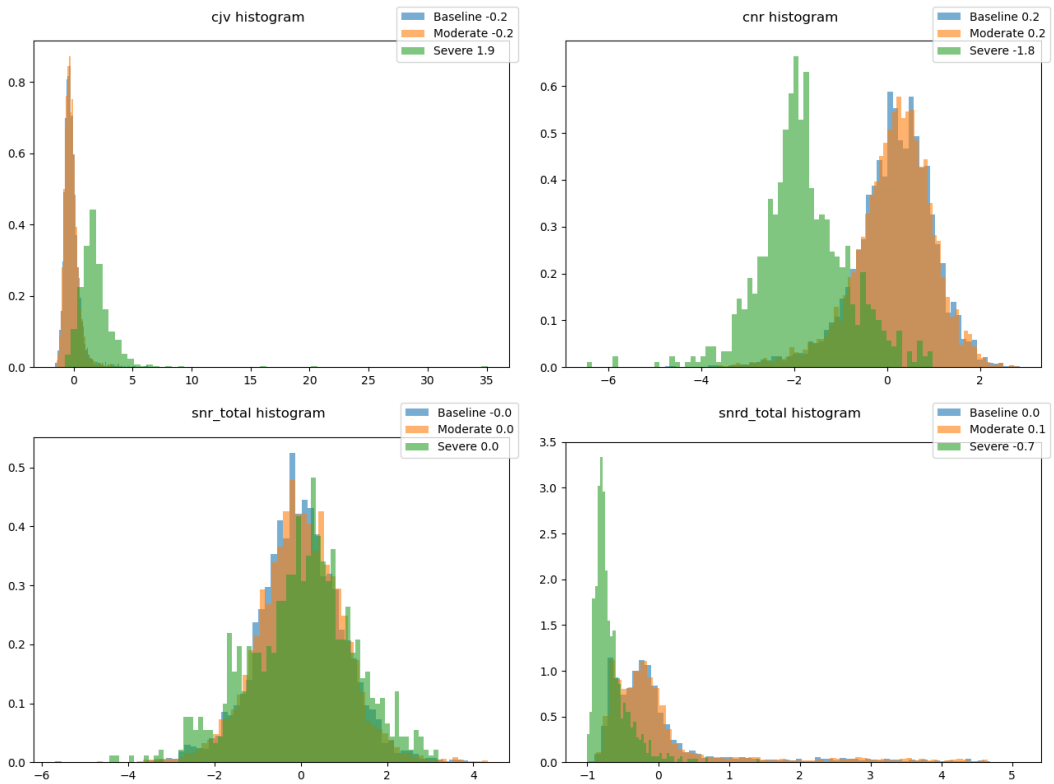

Figure 3.4: Histograms of other potential confounders.

Bibliography

Baranchuk, D., Rubachev, I., Voynov, A., Khrulkov, V., & Babenko, A. (2021). Label-efficient semantic segmentation with diffusion models. *arXiv preprint arXiv:2112.03126*.

Dinsdale, N. K., Jenkinson, M., & Namburete, A. I. (2021). Deep learning-based unlearning of dataset bias for mri harmonisation and confound removal. *NeuroImage*, *228*, 117689.

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, *33*, 6840–6851.

Ho, J., Salimans, T., Gritsenko, A., Chan, W., Norouzi, M., & Fleet, D. J. (2022). Video diffusion models. *arXiv preprint arXiv:2204.03458*.

Kreis, K., Gao, R., & Vahdat, A. (2022). *Denoising diffusion-based generative modeling: Foundations and applications.* Retrieved from https://cvpr2022-tutorial-diffusion-models.github.io/ (Conference on Computer Vision and Pattern Recognition)

Luo, C. (2022). Understanding diffusion models: A unified perspective. *arXiv preprint arXiv:2208.11970*.

Nichol, A., Dhariwal, P., Ramesh, A., Shyam, P., Mishkin, P., McGrew, B., ... Chen, M. (2021). Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*.

Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*.

Saharia, C., Chan, W., Chang, H., Lee, C., Ho, J., Salimans, T., ... Norouzi, M. (2022). Palette: Image-to-image diffusion models. In *Acm siggraph 2022 conference proceedings* (pp. 1–10).

Saharia, C., Ho, J., Chan, W., Salimans, T., Fleet, D. J., & Norouzi, M. (2022). Image super-resolution via iterative refinement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Song, Y., Shen, L., Xing, L., & Ermon, S. (2021). Solving inverse problems in medical imaging with score-based generative models. *arXiv preprint arXiv:2111.08005*.

Turner, A. (2021, Jun). Diffusion models as a kind of vae. *angusturner.github.io*. Retrieved from https://angusturner.github.io/generative_models/2021/06/29/diffusion-probabilistic-models-I.html#citation-3

Weng, L. (2021, Jul). What are diffusion models? *lilianweng.github.io*. Retrieved from https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Zhao, Q., Adeli, E., & Pohl, K. M. (2020). Training confounder-free deep learning models for medical applications. *Nature communications*, *11*(1), 6010.

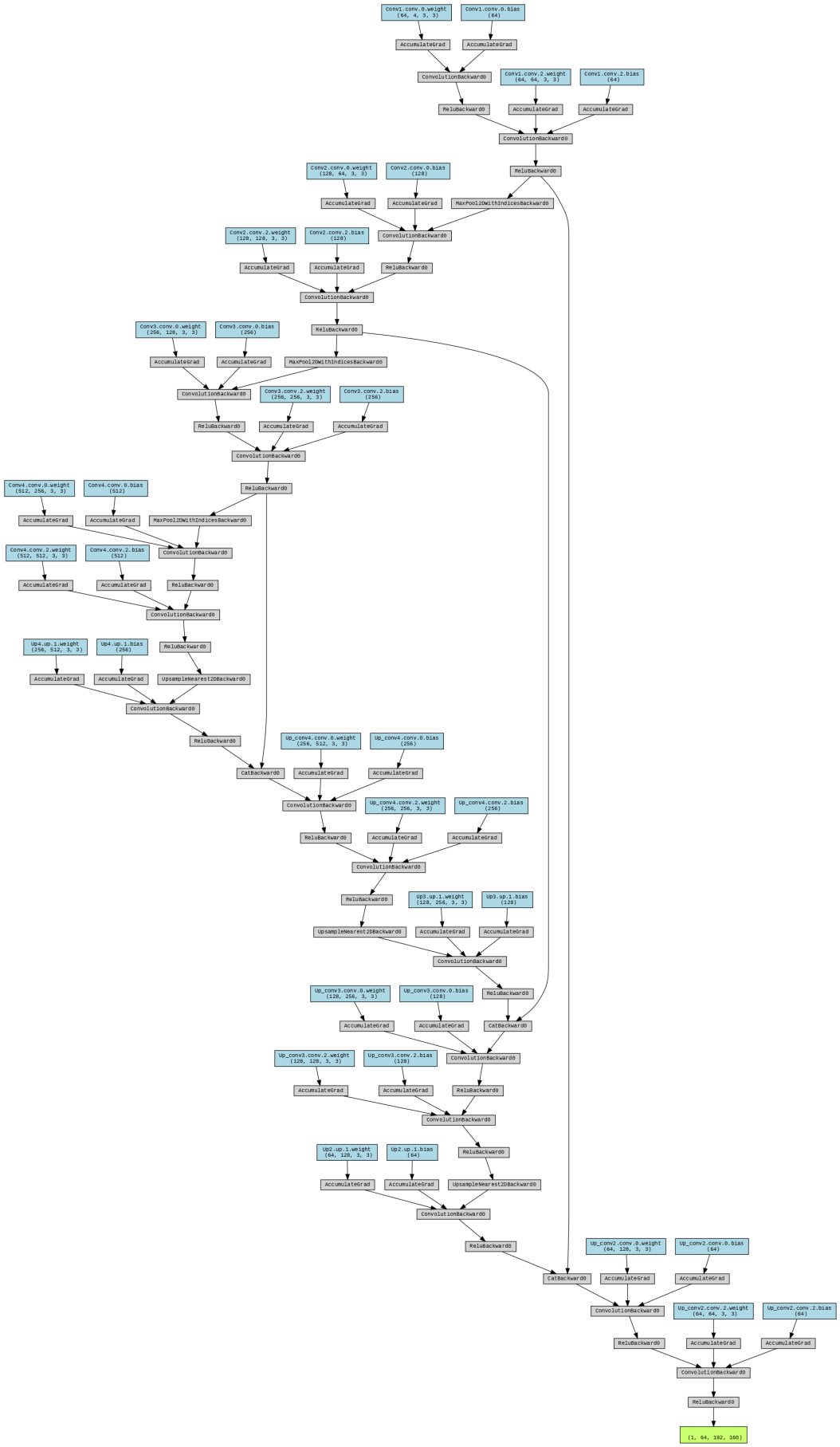

Figure 5: Diagram of architecture of model F.

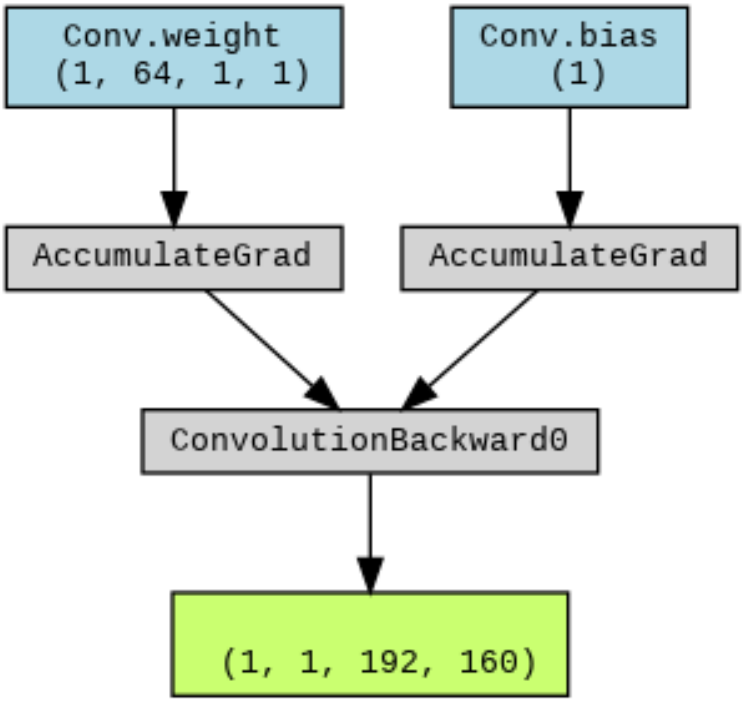

Figure 6: Diagram of architecture of model P.

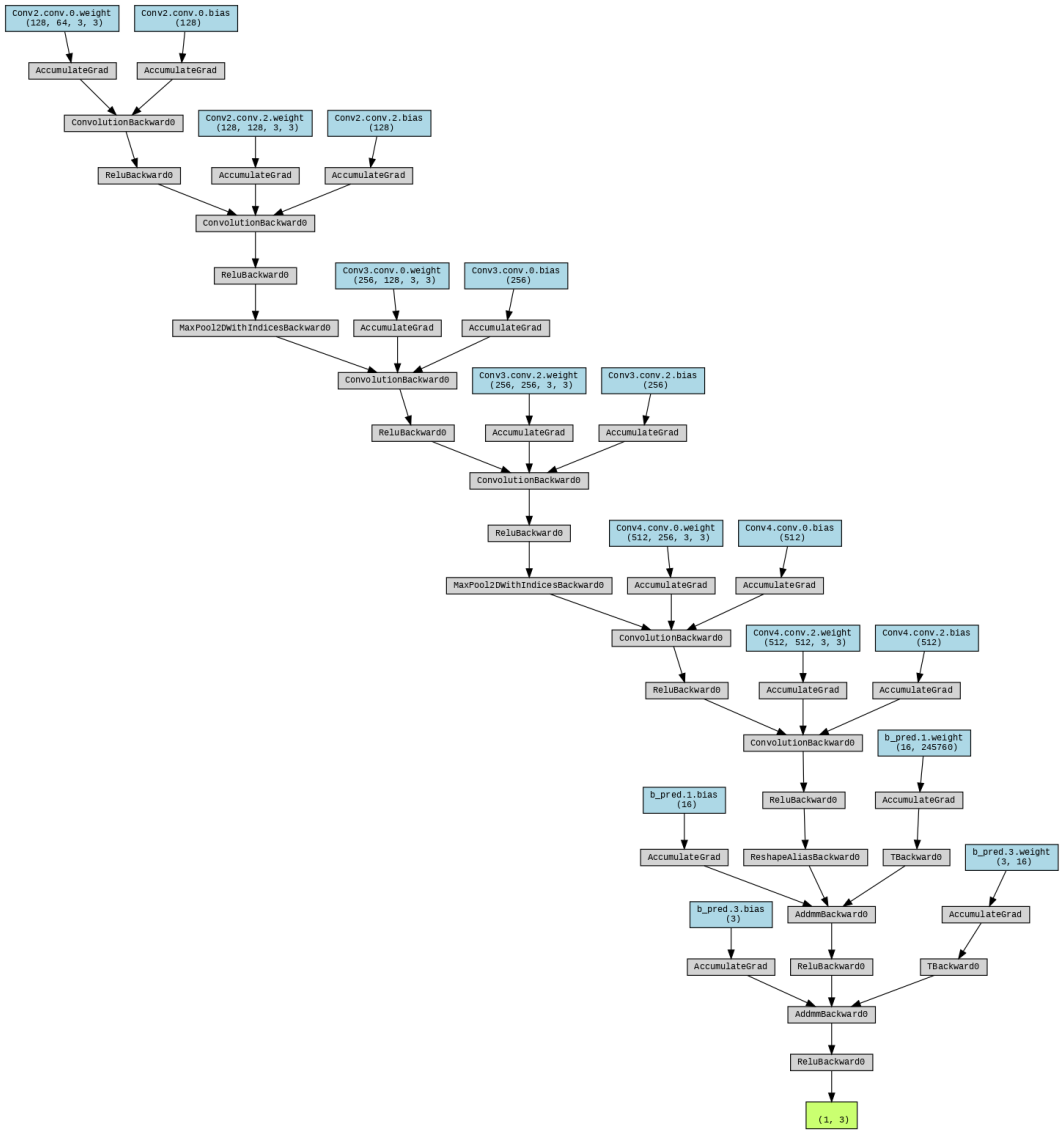

Figure 7: Diagram of architecture of model B.